Les TRIGGERS 2

Délio Duchesne-Brajon — Groupe 1
BTS SIO SLAM — Compte rendu de TP

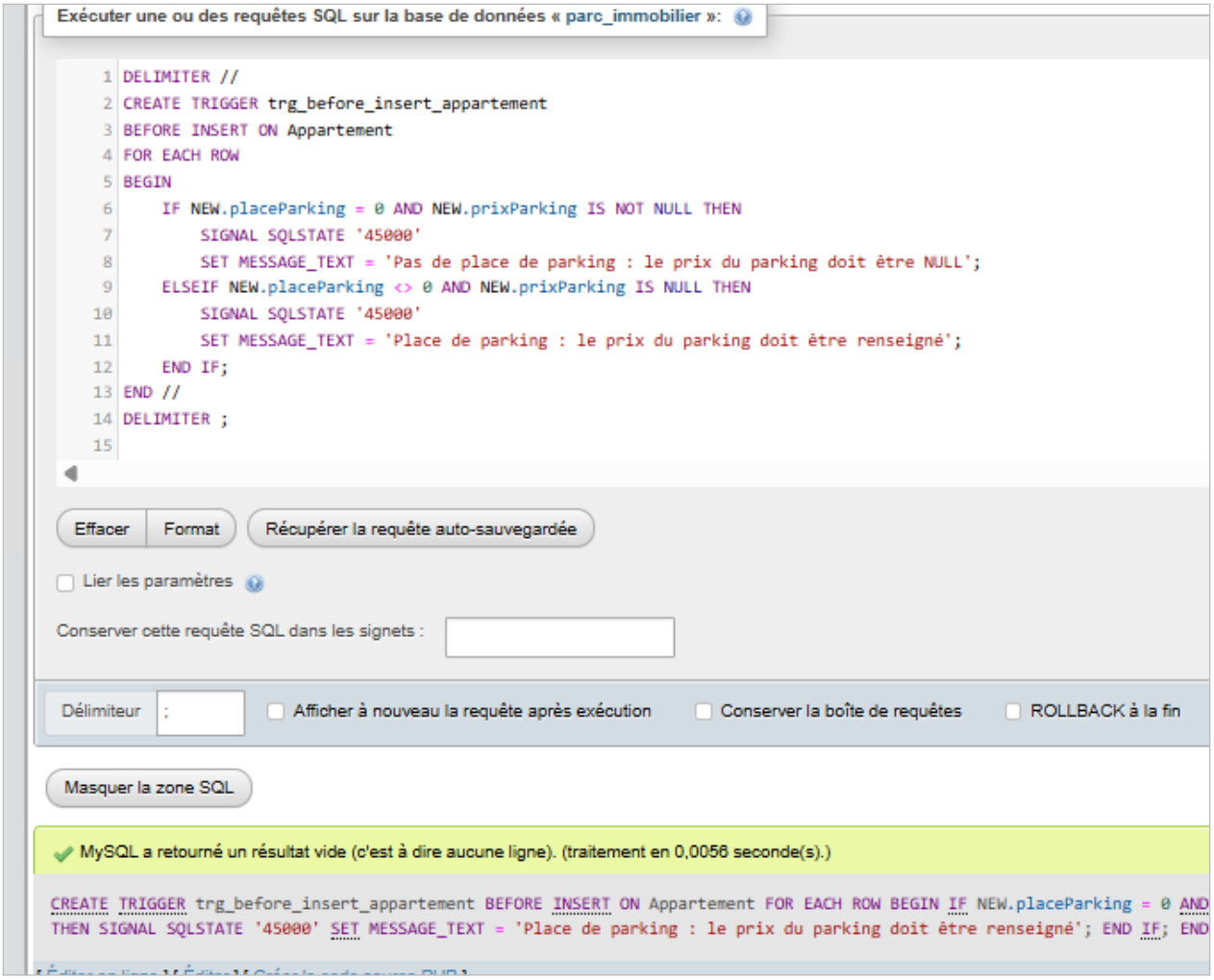
Sommaire

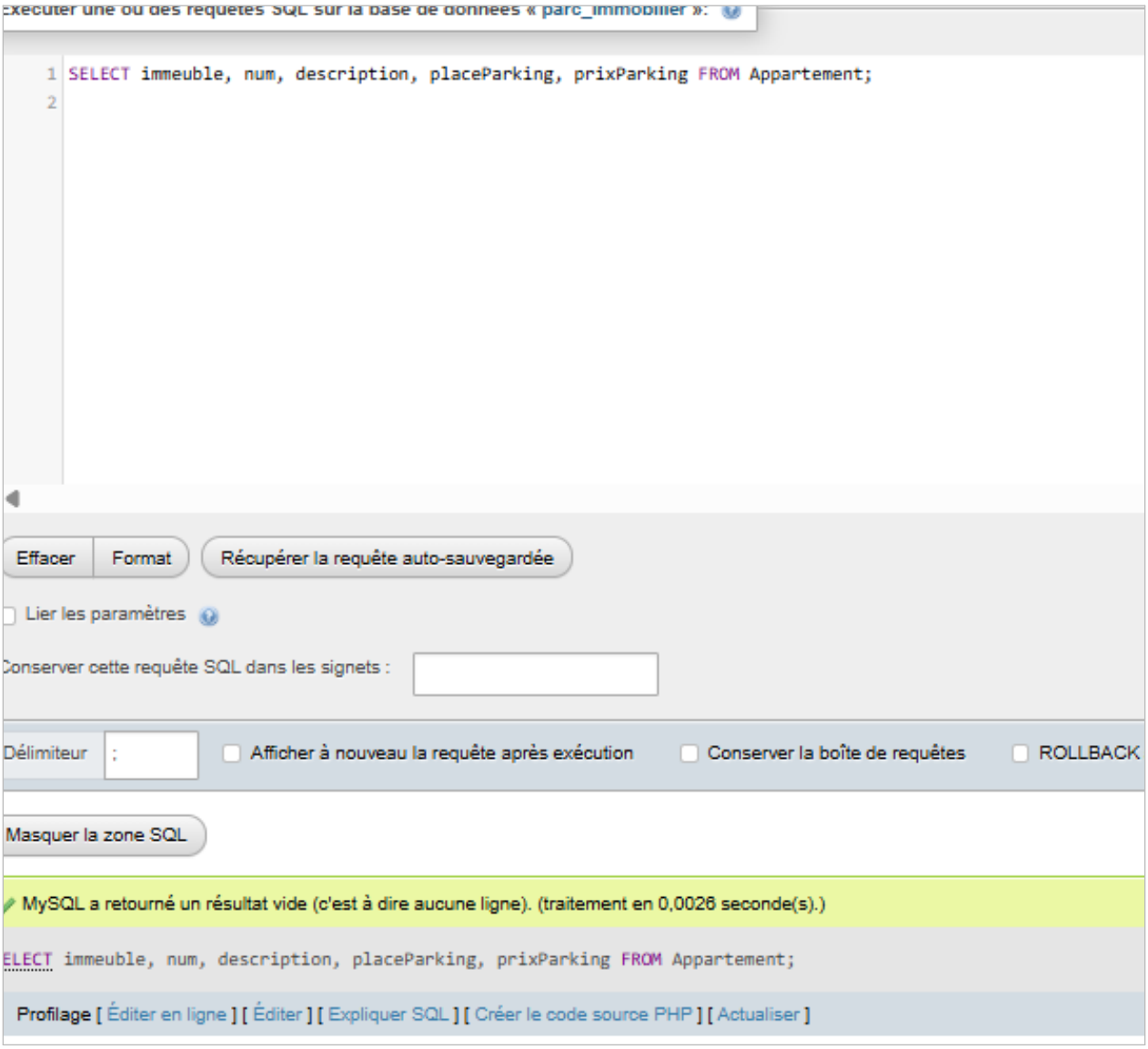
Exercice 1 — Base parc_immobilier 2
Question 1 2
Question 2 3
Question 3 4
Question 4 6
Exercice 2 — Base produits_commandes 8
Question 1 8
Question 2 9
Question 3 9
Question 4 11
Question 5 13
Exercice 3 — Base location_vehicules 15
Question 1 15
Question 2 16
Test 17

Exercice 1 — Base parc_immobilier

Question 1

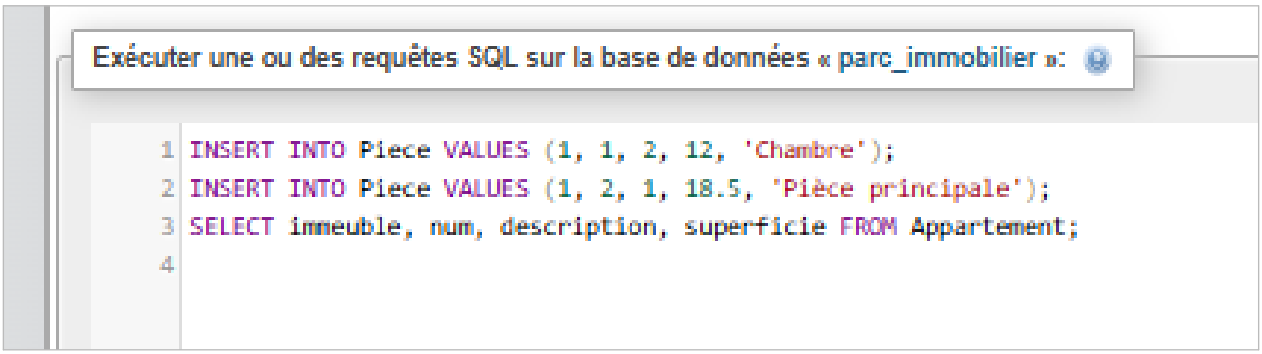
Création du trigger *trg_before_insert_appartement*, qui vérifie avant chaque insertion dans *Appartement* la cohérence entre la place de parking et son prix.

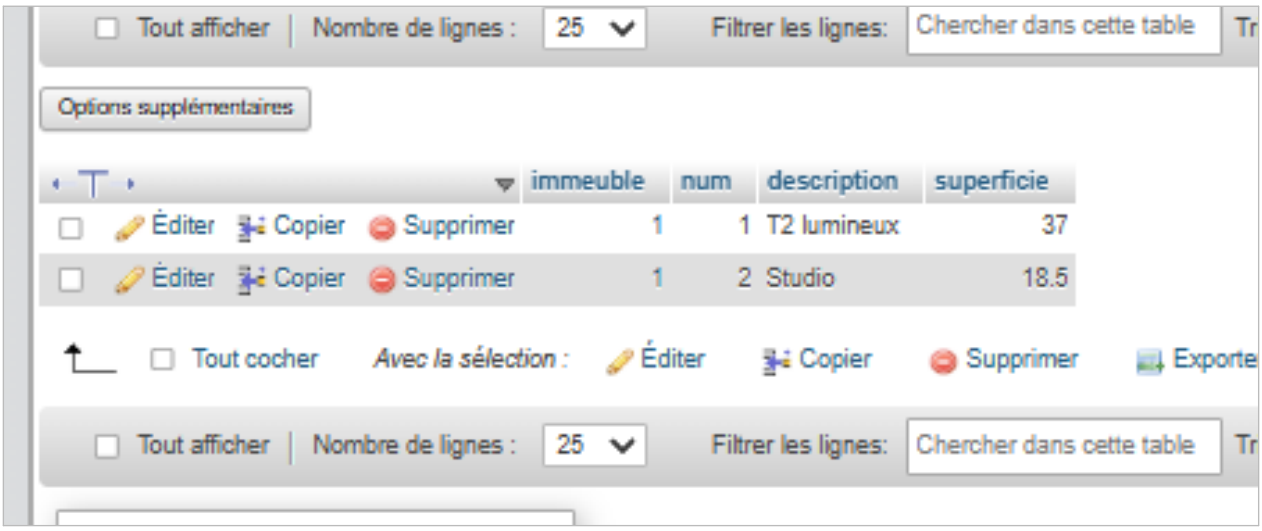




Question 2

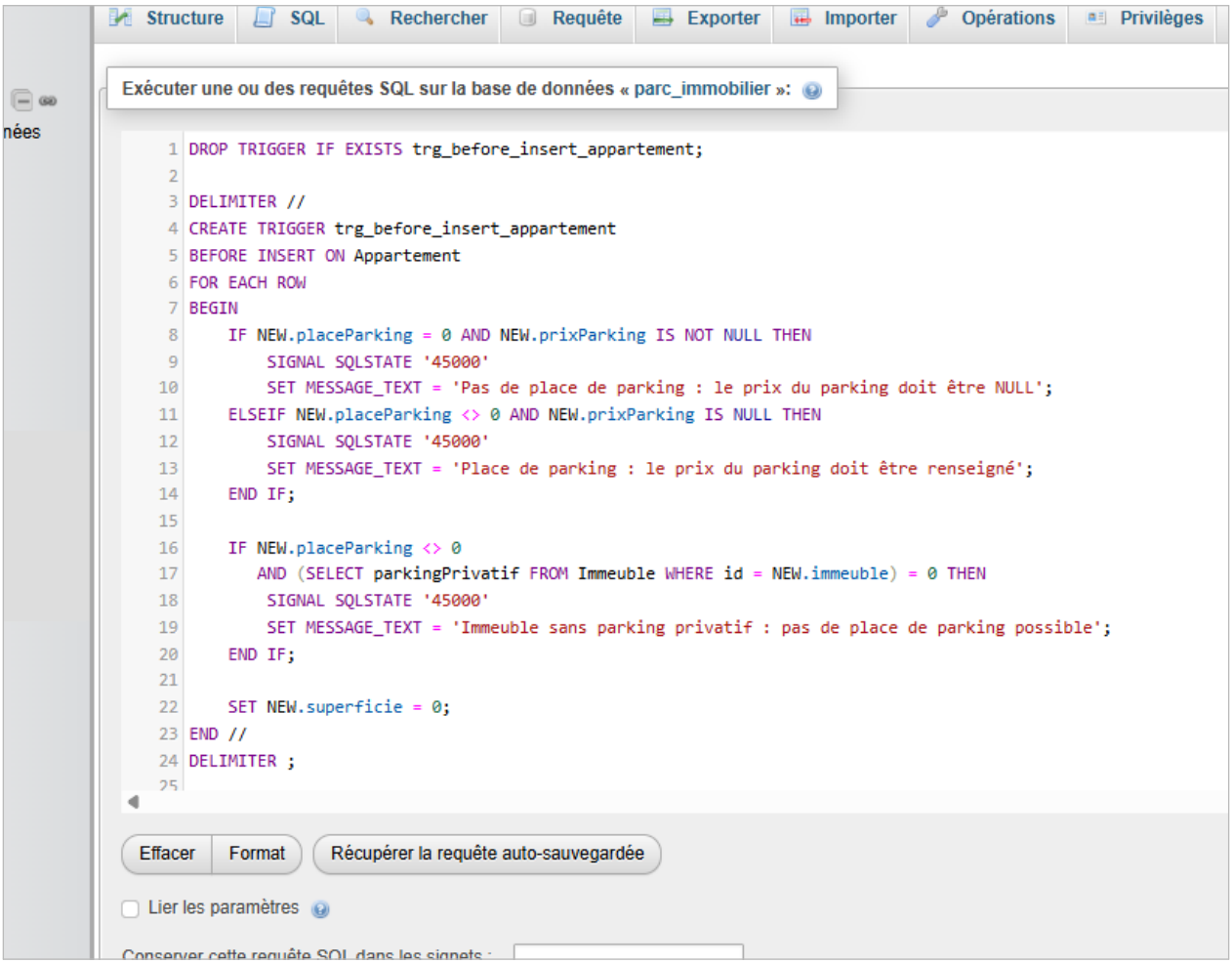
Insertion de deux pièces dans le premier appartement, puis contrôle de la superficie des appartements.

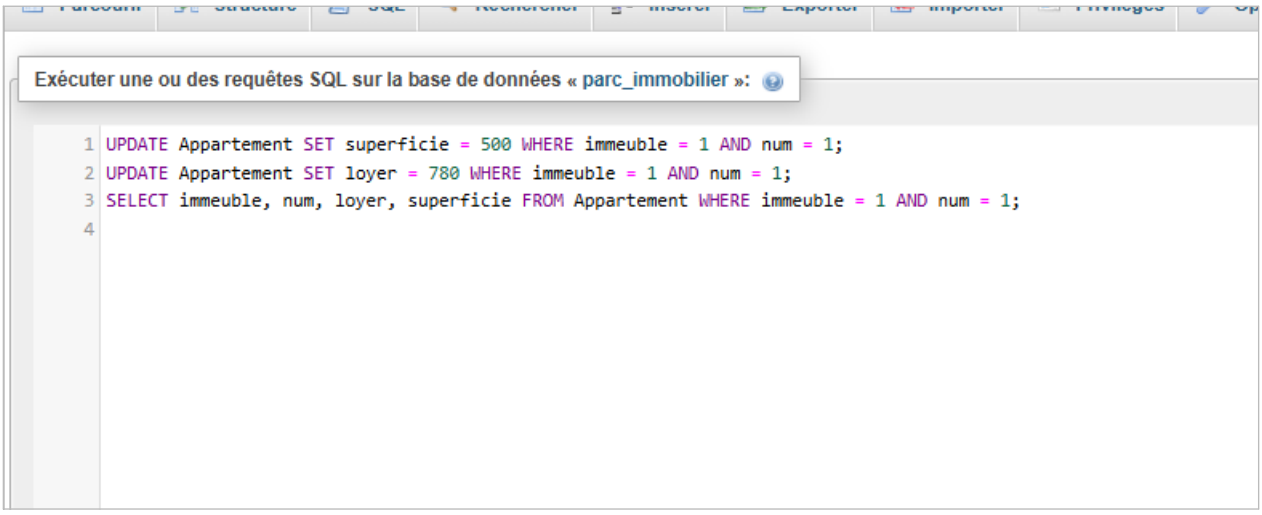


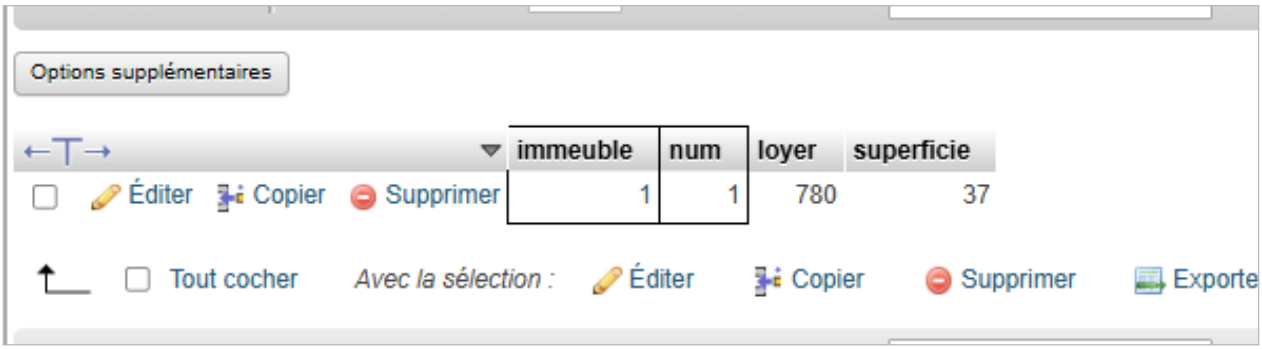


Question 3

Le trigger refuse désormais une place de parking si l'immeuble ne dispose pas d'un parking privatif, et initialise la superficie à 0. Lors du test, la modification directe de la superficie n'est pas prise en compte, contrairement à celle du loyer.

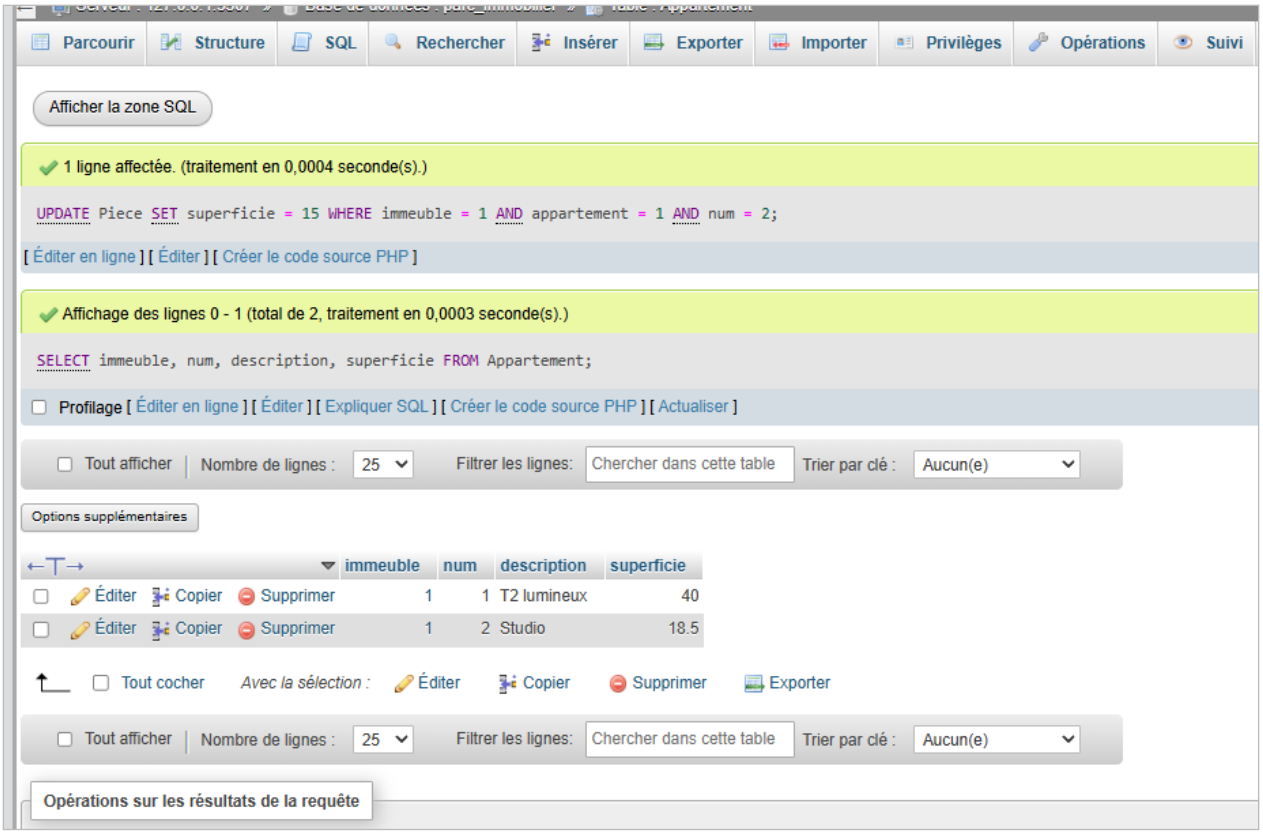


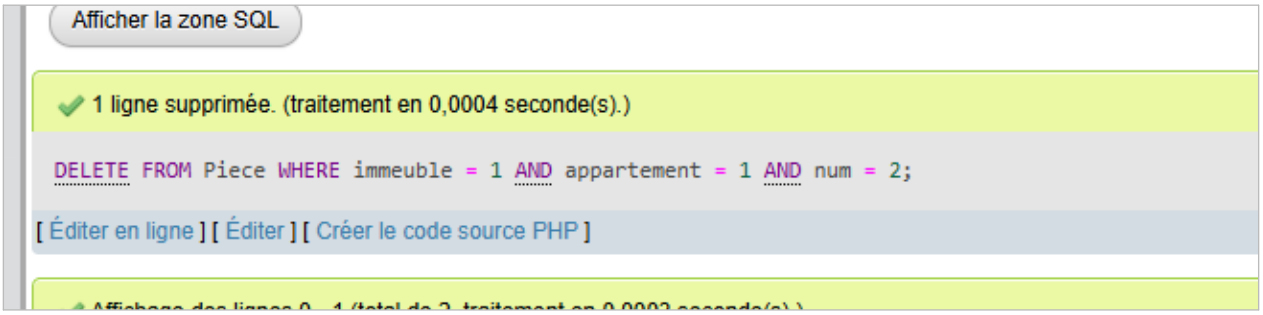

Exécuter une ou des requêtes SQL sur la base de données « parc_immobilier »:
1 UPDATE Appartement SET superficie = 500 WHERE immeuble = 1 AND num = 1;
2 UPDATE Appartement SET loyer = 780 WHERE immeuble = 1 AND num = 1;
3 SELECT immeuble, num, loyer, superficie FROM Appartement WHERE immeuble = 1 AND num = 1;
4

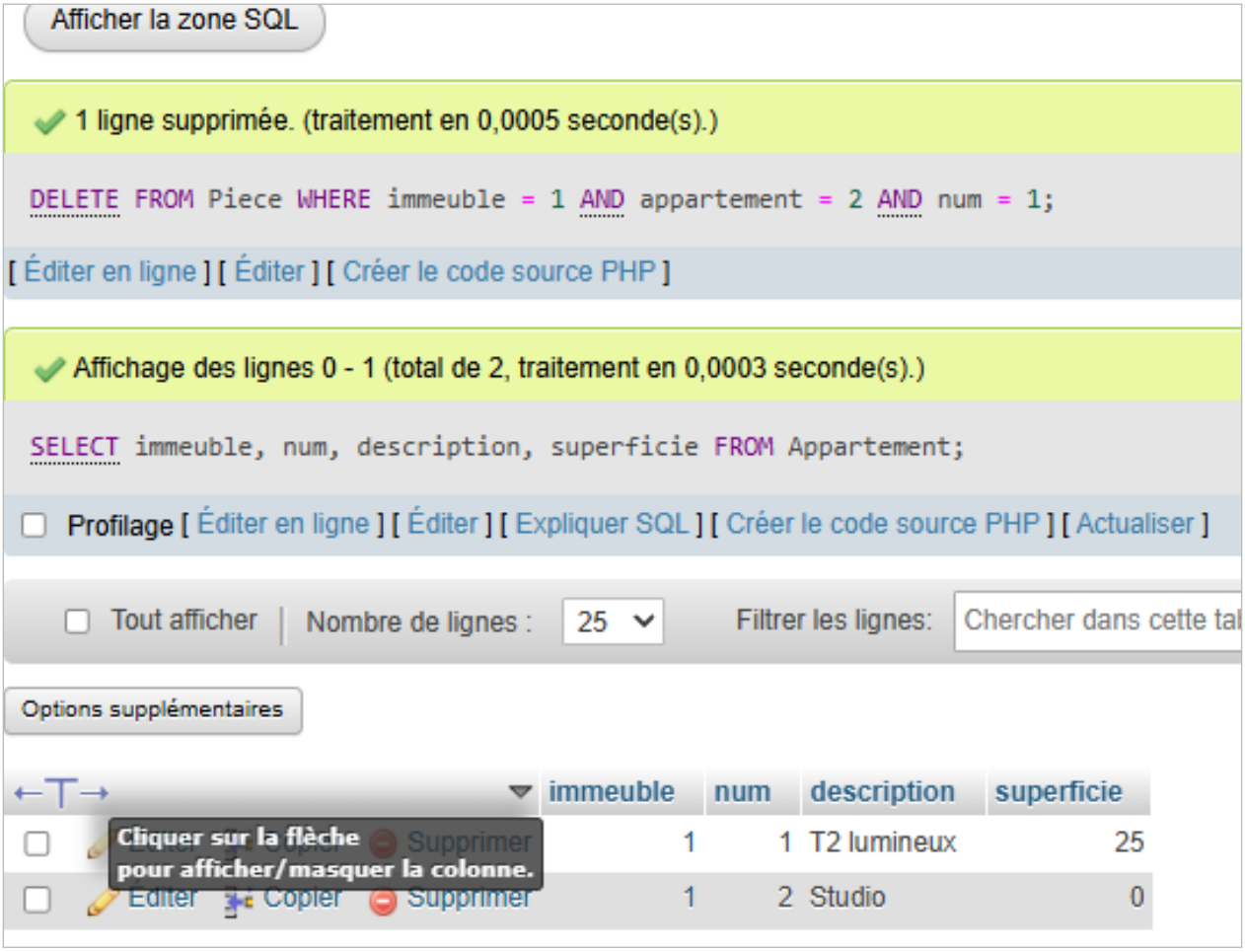

Options supplémentaires
immeuble
num
loyer
superficie
Éditer
Copier
Supprimer
1
1
780
37
Tout cocher
Avec la sélection :
Éditer
Copier
Supprimer
Exporte

Question 4

Tests de modification puis de suppression de pièces : la superficie de l'appartement est recalculée automatiquement après chaque opération.






Afficher la zone SQL
1 ligne supprimée. (traitement en 0,0005 seconde(s).)
DELETE FROM Piece WHERE immeuble = 1 AND appartement = 2 AND num = 1;
[Éditer en ligne] [Éditer] [Créer le code source PHP]
Affichage des lignes 0 - 1 (total de 2, traitement en 0,0003 seconde(s).)
SELECT immeuble, num, description, superficie FROM Appartement;
Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]
Tout afficher
Nombre de lignes : 25
Filtrer les lignes: Chercher dans cette tal
Options supplémentaires
immeuble
num
description
superficie
Cliquer sur la flèche pour afficher/masquer la colonne.
Supprimer
1 1 T2 lumineux 25
Éditer
Copier
Supprimer
1 2 Studio 0

Exercice 2 — Base produits_commandes

Question 1

Création de la base et des tables.

```
CREATE DATABASE produits_commandes;
USE produits_commandes;

CREATE TABLE Produit(
    code INT(11) NOT NULL PRIMARY KEY,
    libelle VARCHAR(100) NOT NULL,
    prix DOUBLE NOT NULL,
    formation TINYINT(1) NOT NULL,
    poids INT(11)
);

CREATE TABLE Mode_Livraison(
    num INT(11) NOT NULL PRIMARY KEY,
    libelle VARCHAR(50) NOT NULL
);

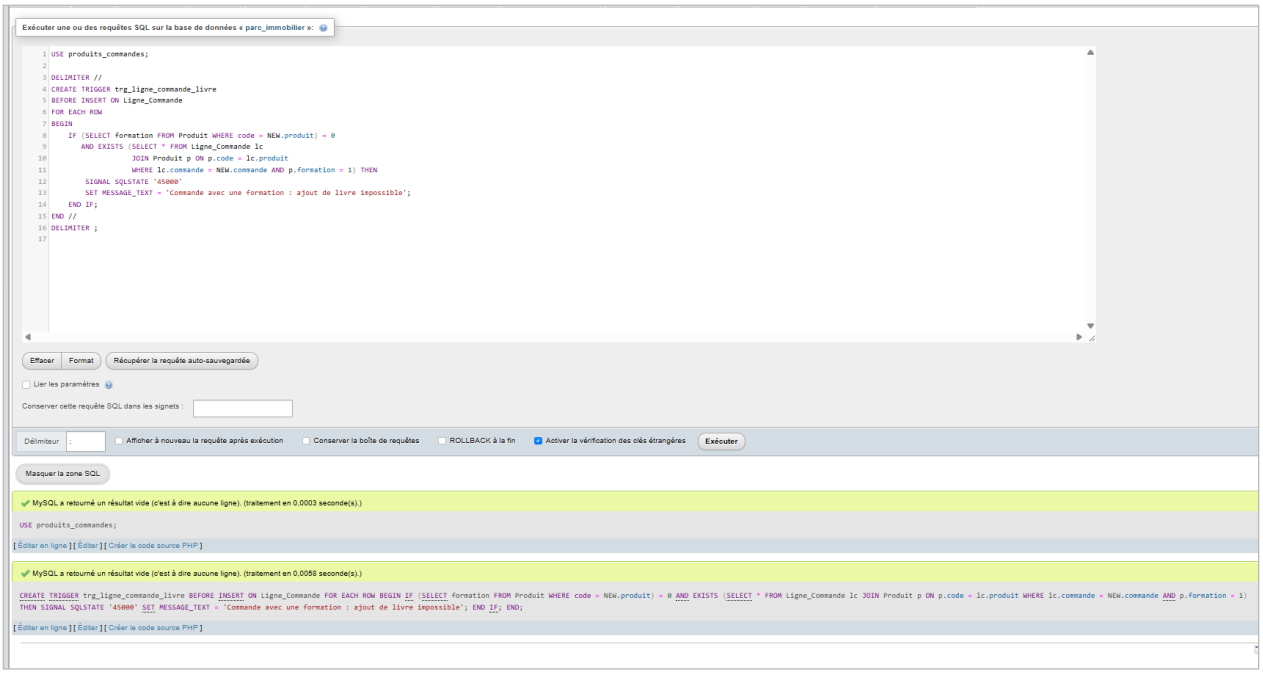
CREATE TABLE Frais_Transport(
    livraison INT(11), num INT(11) NOT NULL,
    prix DOUBLE NOT NULL, poidsMin INT(11) NOT NULL, poidsMax INT(11) NOT NULL,
    CONSTRAINT pk_frais_transport PRIMARY KEY (livraison, num),
    CONSTRAINT fk_mode_livraison FOREIGN KEY (livraison) REFERENCES Mode_Livraison(num)
);

CREATE TABLE Commande(
    num INT(11) NOT NULL PRIMARY KEY,
    destinataire VARCHAR(100) NOT NULL,
    prixTotal DOUBLE NOT NULL DEFAULT 0,
    livraison INT(11), transport INT(11),
    CONSTRAINT fk_frais_transport
    FOREIGN KEY (livraison, transport) REFERENCES Frais_Transport(livraison, num)
);

CREATE TABLE Ligne_Commande(
    commande INT(11), produit INT(11),
    prixu DOUBLE NOT NULL, quantite INT(11) NOT NULL, prixt DOUBLE NOT NULL,
    CONSTRAINT pk_ligne_commande PRIMARY KEY (commande, produit),
    CONSTRAINT fk_commande FOREIGN KEY (commande) REFERENCES Commande(num),
    CONSTRAINT fk_produit FOREIGN KEY (produit) REFERENCES Produit(code)
);
```

Question 2

Trigger *trg_ligne_commande_livre* : il empêche l'ajout d'un livre dans une commande qui contient déjà une formation.

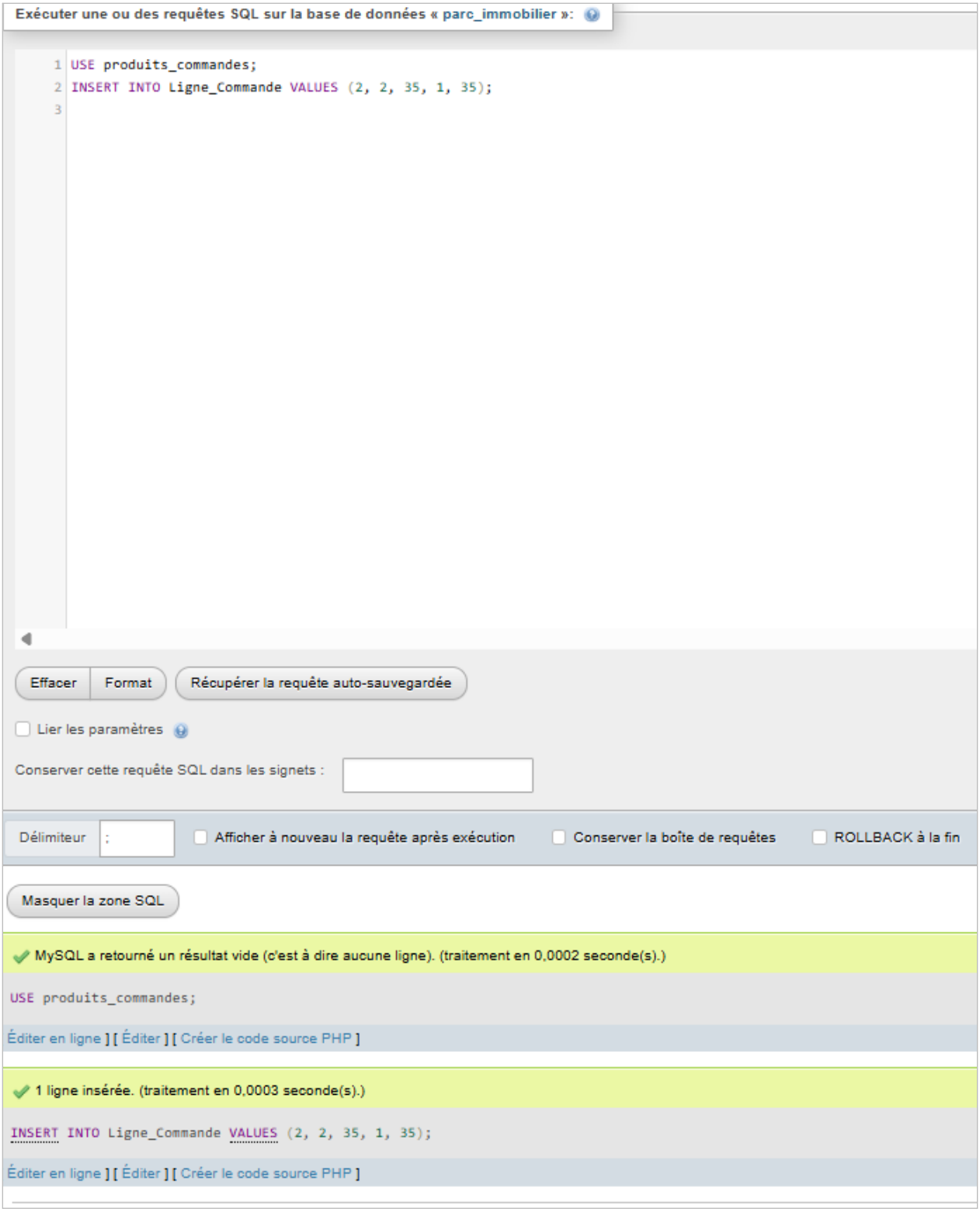


Question 3

Trigger *trg_ligne_commande_formation* : il empêche l'ajout d'une formation dans une commande qui contient déjà des livres, puis test d'insertion.

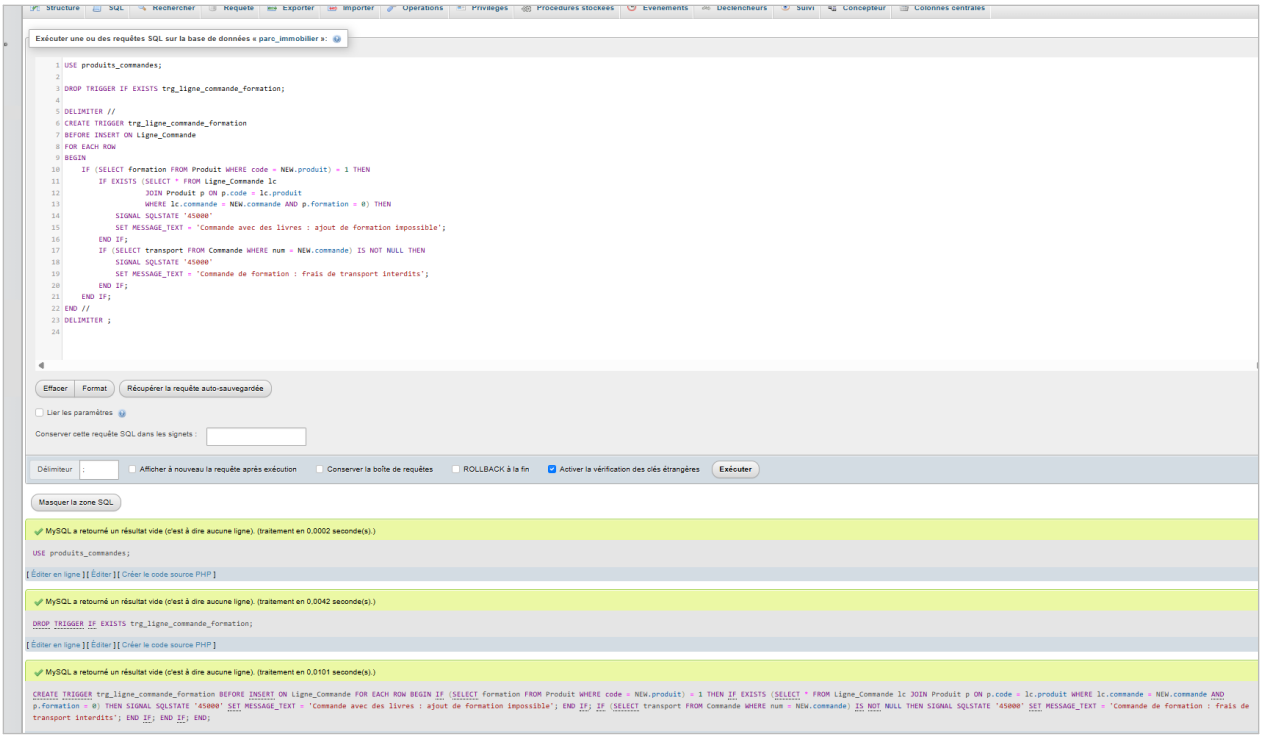
```
USE produits_commandes;

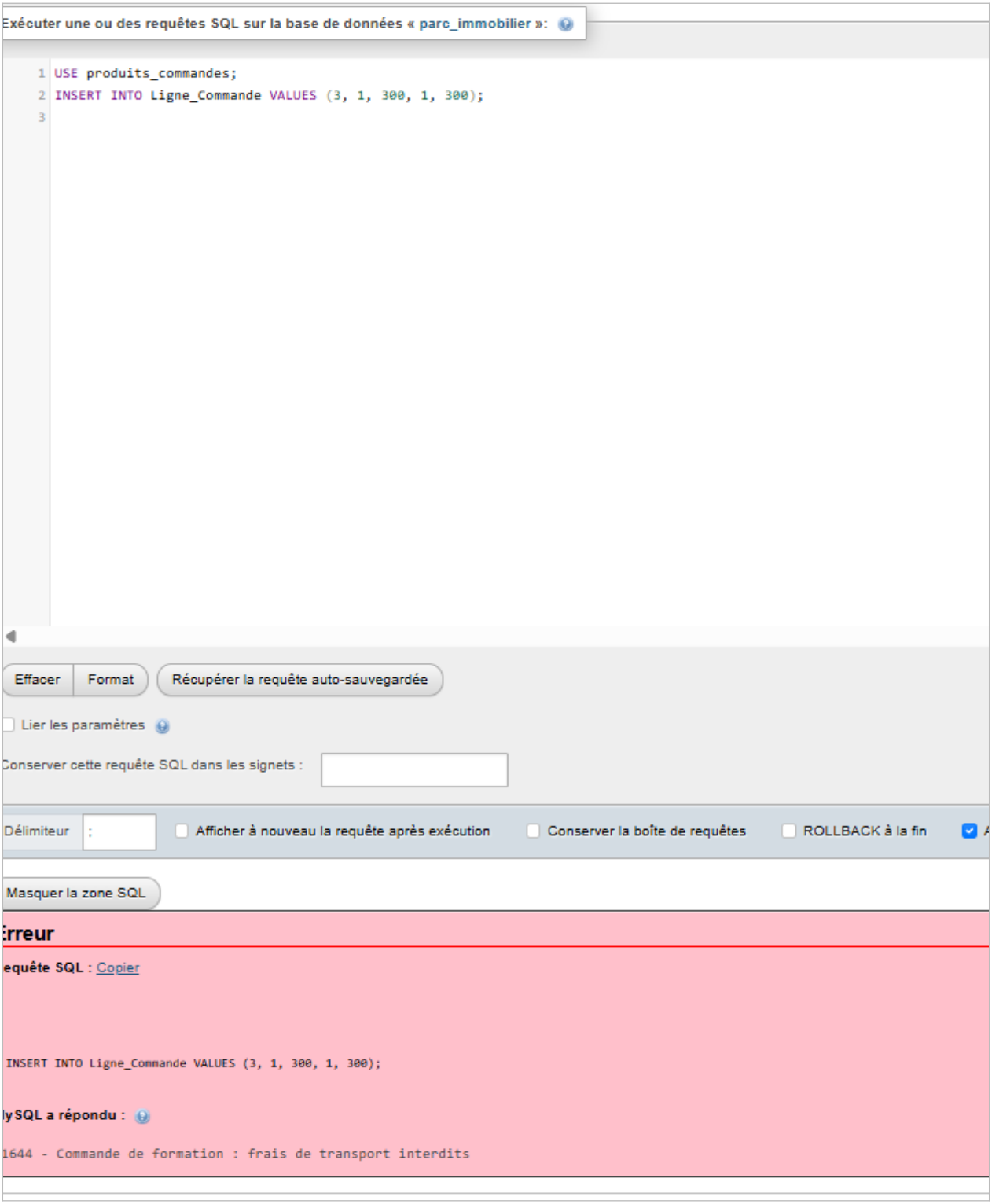
DELIMITER //
CREATE TRIGGER trg_ligne_commande_formation
BEFORE INSERT ON Ligne_Commande
FOR EACH ROW
BEGIN
    IF (SELECT formation FROM Produit WHERE code = NEW.produit) = 1
       AND EXISTS (SELECT * FROM Ligne_Commande lc
                   JOIN Produit p ON p.code = lc.produit
                   WHERE lc.commande = NEW.commande AND p.formation = 0) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Commande avec des livres : ajout de formation impossible';
    END IF;
END //
DELIMITER ;
```


Exécuter une ou des requêtes SQL sur la base de données « parc_immobilier »:
1 USE produits_commandes;
2 INSERT INTO Ligne_Commande VALUES (2, 2, 35, 1, 35);
3
Effacer
Format
Récupérer la requête auto-sauvegardée
Lier les paramètres
Conserver cette requête SQL dans les signets :
Délimiteur ;
Afficher à nouveau la requête après exécution
Conserver la boîte de requêtes
ROLLBACK à la fin
Masquer la zone SQL
MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0002 seconde(s).)
USE produits_commandes;
Éditer en ligne] [Éditer] [Créer le code source PHP]
1 ligne insérée. (traitement en 0,0003 seconde(s).)
INSERT INTO Ligne_Commande VALUES (2, 2, 35, 1, 35);
Éditer en ligne] [Éditer] [Créer le code source PHP]

Question 4

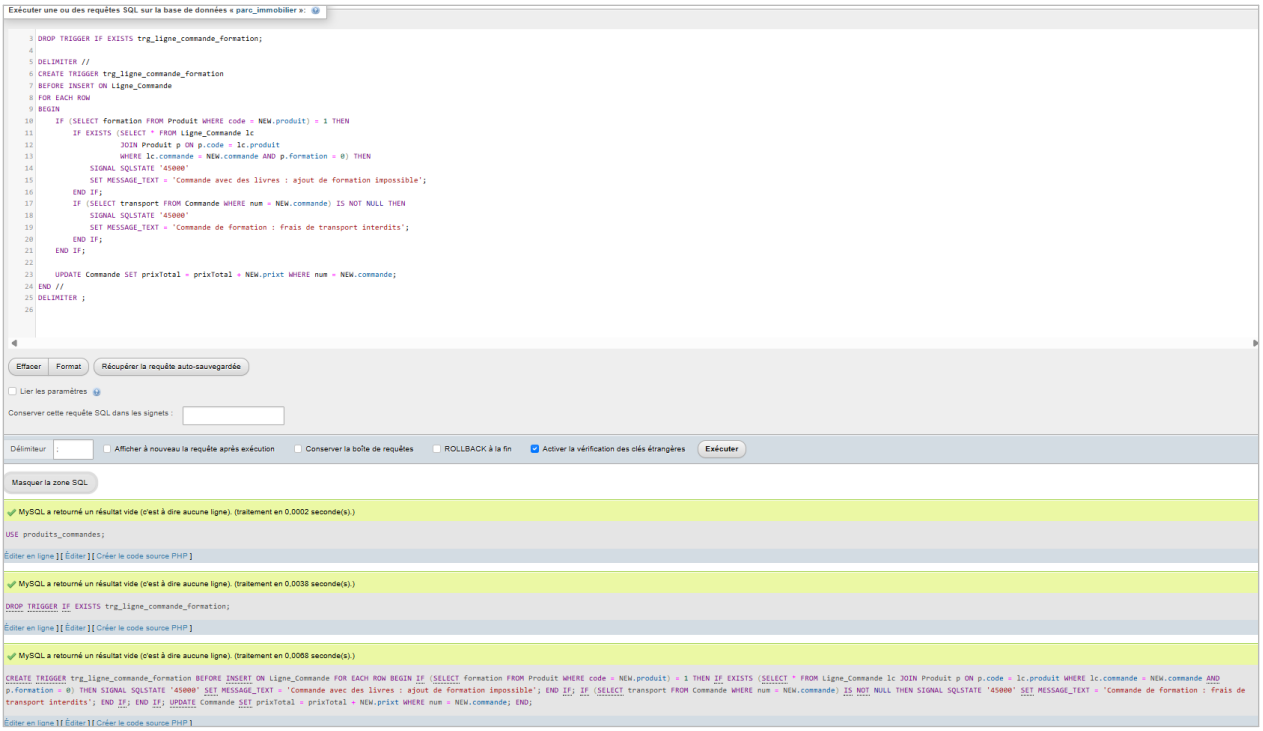
Ajout d'un second contrôle : une commande de formation ne peut pas comporter de frais de transport. L'insertion de test est bien rejetée.

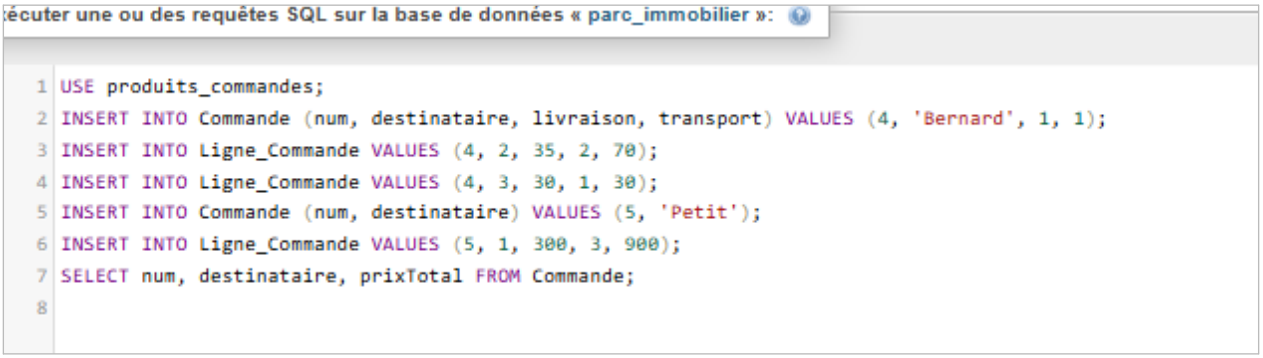



Exécuter une ou des requêtes SQL sur la base de données « parc_immobilier »:
1 USE produits_commandes;
2 INSERT INTO Ligne_Commande VALUES (3, 1, 300, 1, 300);
3
Effacer
Format
Récupérer la requête auto-sauvegardée
Lier les paramètres
Conserver cette requête SQL dans les signets :
Délimiteur
;
Afficher à nouveau la requête après exécution
Conserver la boîte de requêtes
ROLLBACK à la fin
Masquer la zone SQL
rreur
equête SQL : Copier
INSERT INTO Ligne_Commande VALUES (3, 1, 300, 1, 300);
ySQL a répondu :
1644 - Commande de formation : frais de transport interdits

Question 5

Le trigger met également à jour le prix total de la commande à chaque ligne ajoutée. La vérification porte sur les commandes 4 et 5.






Options supplémentaires
num
destinataire
prixTotal
Éditer Copier Supprimer 1 Dupont 0
Éditer Copier Supprimer 2 Martin 0
Éditer Copier Supprimer 3 Durand 0
Éditer Copier Supprimer 4 Bernard 100
Éditer Copier Supprimer 5 Petit 900
Tout cocher
Avec la sélection :
Éditer
Copier
Suppr

Exercice 3 — Base location_vehicules

Question 1

Création de la base et des tables.

Exécuter une ou des requêtes SQL sur la base de données « parc_immobilier »:

```sql
CREATE DATABASE location_vehicules;
USE location_vehicules;

CREATE TABLE Permis(
 num INT(11) NOT NULL PRIMARY KEY,
 type VARCHAR(5) NOT NULL
);

CREATE TABLE Conducteur(
 num INT(11) NOT NULL PRIMARY KEY,
 civilite VARCHAR(5) NOT NULL, prenom VARCHAR(50) NOT NULL, nom VARCHAR(50) NOT NULL
);

CREATE TABLE Vehicule(
 num INT(11) NOT NULL PRIMARY KEY,
 immatriculation VARCHAR(10) NOT NULL,
 permis INT(11) NOT NULL,
 CONSTRAINT fk_vehicule_permis FOREIGN KEY (permis) REFERENCES Permis(num)
);

CREATE TABLE Titulaire(
 conducteur INT(11), permis INT(11),
 CONSTRAINT pk_titulaire PRIMARY KEY (conducteur, permis),
 CONSTRAINT fk_titulaire_conducteur FOREIGN KEY (conducteur) REFERENCES Conducteur(num),
 CONSTRAINT fk_titulaire_permis FOREIGN KEY (permis) REFERENCES Permis(num)
);

CREATE TABLE Periode(
 debut DATETIME NOT NULL, fin DATETIME NOT NULL,
 CONSTRAINT pk_periode PRIMARY KEY (debut, fin)
);

CREATE TABLE Location(
 vehicule INT(11), debut DATETIME, fin DATETIME,
 conducteur INT(11) NOT NULL,
 CONSTRAINT pk_location PRIMARY KEY (vehicule, debut, fin),
 CONSTRAINT fk_location_vehicule FOREIGN KEY (vehicule) REFERENCES Vehicule(num),
 CONSTRAINT fk_location_periode FOREIGN KEY (debut, fin) REFERENCES Periode(debut, fin),
 CONSTRAINT fk_location_conducteur FOREIGN KEY (conducteur) REFERENCES Conducteur(num)
);

```

Question 2

Triggers *trg_before_insert_location* et *trg_before_update_location* : ils vérifient que le conducteur possède le permis requis pour le véhicule loué.

```sql
USE location_vehicules;

DELIMITER //
CREATE TRIGGER trg_before_insert_location
BEFORE INSERT ON Location
FOR EACH ROW
BEGIN
    IF NOT EXISTS (SELECT * FROM Titulaire t
                   JOIN Vehicule v ON v.permis = t.permis
                   WHERE v.num = NEW.vehicule AND t.conducteur = NEW.conducteur) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Le conducteur ne possède pas le permis requis pour ce véhicule';
    END IF;
END //
DELIMITER ;

DELIMITER //
CREATE TRIGGER trg_before_update_location
BEFORE UPDATE ON Location
FOR EACH ROW
BEGIN
    IF NOT EXISTS (SELECT * FROM Titulaire t
                   JOIN Vehicule v ON v.permis = t.permis
                   WHERE v.num = NEW.vehicule AND t.conducteur = NEW.conducteur) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Le conducteur ne possède pas le permis requis pour ce véhicule';
    END IF;
END //
DELIMITER ;

```

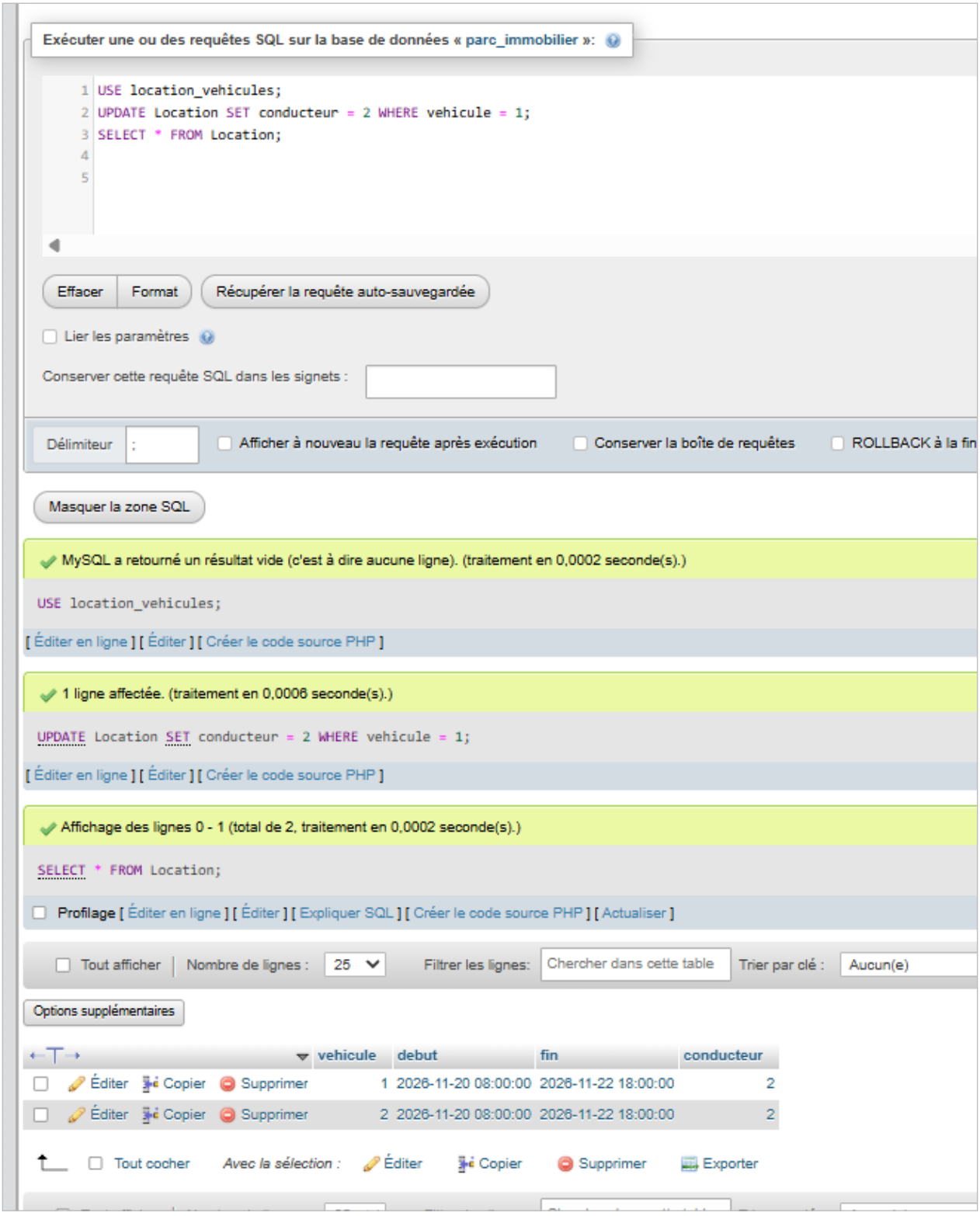
Test

Insertion d'un jeu de données, puis modification d'une location. Le conducteur choisi possède le permis requis, la mise à jour est donc acceptée.

Exécuter une ou des requêtes SQL sur la base de données « parc_immobilier »:

```sql
USE location_vehicules;
INSERT INTO Permis VALUES (1, 'A'), (2, 'B'), (3, 'C');
INSERT INTO Conducteur VALUES (1, 'M.', 'Paul', 'Martin'), (2, 'Mme', 'Julie', 'Durand');
INSERT INTO Titulaire VALUES (1, 2), (2, 2), (2, 3);
INSERT INTO Vehicule VALUES (1, 'AB-123-CD', 2), (2, 'EF-456-GH', 3);
INSERT INTO Periode VALUES ('2026-11-20 08:00:00', '2026-11-22 18:00:00');

```


Exécuter une ou des requêtes SQL sur la base de données « parc_immobilier »:
USE location_vehicules;
UPDATE Location SET conducteur = 2 WHERE vehicule = 1;
SELECT * FROM Location;
Effacer
Format
Récupérer la requête auto-sauvegardée
Lier les paramètres
Conserver cette requête SQL dans les signets :
Délimiteur
;
Afficher à nouveau la requête après exécution
Conserver la boîte de requêtes
ROLLBACK à la fin
Masquer la zone SQL
MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0002 seconde(s).)
USE location_vehicules;
[Éditer en ligne] [Éditer] [Créer le code source PHP]
1 ligne affectée. (traitement en 0,0006 seconde(s).)
UPDATE Location SET conducteur = 2 WHERE vehicule = 1;
[Éditer en ligne] [Éditer] [Créer le code source PHP]
Affichage des lignes 0 - 1 (total de 2, traitement en 0,0002 seconde(s).)
SELECT * FROM Location;
Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]
Tout afficher
Nombre de lignes :
25
Filtrer les lignes:
Chercher dans cette table
Trier par clé :
Aucun(e)
Options supplémentaires
vehicule
debut
fin
conducteur
Éditer
Copier
Supprimer
1
2026-11-20 08:00:00
2026-11-22 18:00:00
2
Éditer
Copier
Supprimer
2
2026-11-20 08:00:00
2026-11-22 18:00:00
2
Tout cocher
Avec la sélection :
Éditer
Copier
Supprimer
Exporter